Aprendizagem de Máquina

Aula 04 – Pré-processamento sem Vazamento e Pipelines

Prof. Dr. Jonas Krause

PUCPR – PPGCIS – Ciência da Computação

27 de agosto de 2026

A Engenharia da Prevenção em Machine Learning

Aula 04: Treino, Teste e o Combate ao Vazamento de Dados

PROJECT SPECIFICATION
[LEGEND]

PROF. DR. JONAS KRAUSE | PUCPR - PPGCIS

DATE: 2024-10-27 | REV: 1.0 | SCHEMATIC NO: 04.01

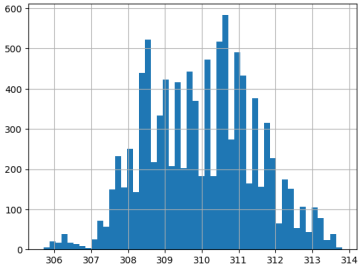
Gemini Notebook

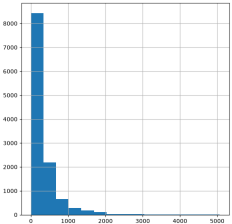
Atividades 1, 2 e 3 – equipes e alunos

Entre parênteses estão as atividades concluídas, conforme o nome das pastas no ZIP.

Eq01 (1, 2, 3)	Emily Pontes; Erick Maestri; Julia Kociolek; Sophia
Eq02 (1, 2, 3)	Adryan Costa Silva; Hassan Ali El Gazouini; Hussein Ali El Gazouini; Victor Augusto Esmaniotto
Eq04 (1, 2, 3)	Renan da Silva Oliveira Andrade; Renan Americo Herculano; Lucas da Costa Paula; Guilherme Arcanjo de Morais
Eq06 (1, 2)	Gustavo Klinfuss da Silva
Eq07 (1, 2, 3)	Alisson Daledone Hass de Lima; Geandro Marques da Silva; Juliano Teles Tolomini Biasotto
Eq08 (1, 2, 3)	Artur Moretti Zimmermann; Bruno Navarro Ivaitiuk; Gustavo Lisboa; Joaquim Faraco; Vinicius Souza Penhorato
Eq09 (1, 2, 3)	Alana da Conceição Queiroz; Leticia Maria Maia de Andrade Vieira; Larissa Alves da Silva; Thomas Manussadjian Steinhausser
Eq10 (1)	Emmanuel Antonietti Ribeiro dos Santos; Enzo Wasko Amorim; Juliano Cesar Enns Miranda Marcos; Pedro Joslin Cavalli; Vinícius Paraíso Dias
Eq11 (1, 2, 3)	Ricardo Lucas Kucek; Lucas Ferraz dos Santos; Riscala Miguel Fadel Neto; Pedro Senes Velloso Ribeiro
Eq12 (1, 2)	Ana Sales
Eq13 (1, 2, 3)	Murilo Zimerman Fortaleza
Eq14 (1, 2, 3)	André Vinícius; Felipe Willian; Gabriel Calado; Luis Gustavo
Eq15 (1, 2, 3)	Alan Filipe; Antonio Bernardo; Gustavo Lona; Pedro Henrique
Eq16 (1, 2, 3)	Brenda Barbosa; Eros dos Santos; Natalia Trevisol
Eq17 (1, 2, 3)	Bruno Mussi
Eq18 (1, 2, 3)	Ana Carolina Afonso Meiado; Ana Carolina Curi de Sales; Anna Bosquillia Navarro

O que os gráficos indicam sobre os dados?





Exemplo mais adequado

A distribuição está relativamente concentrada e não apresenta uma cauda extrema. A média e o desvio padrão representam melhor a região central dos dados.

Exemplo problemático

A distribuição apresenta forte assimetria e uma cauda longa. Poucos valores podem dominar a transformação; investigue outliers e considere transformação logarítmica ou `RobustScaler`.

Antes de normalizar, dividimos treino e teste

Conjunto de treino

É usado para aprender:

- parâmetros das transformações;
- média, desvio, mínimo e máximo;
- padrões utilizados pelo modelo.

Conjunto de teste

Representa dados ainda desconhecidos:

- não participa do treinamento;
- não pode definir transformações;
- é usado para a avaliação final.

O que faremos nesta aula?

1. dividir os dados em treino e teste;
2. ajustar a normalização somente no treino;
3. aplicar a mesma transformação ao treino e ao teste;
4. treinar o modelo e avaliar seu desempenho no teste.

Regra principal

O teste pode ser transformado, mas nunca pode ser usado para aprender a transformação.

Normalizar treino e teste sem misturar informações

A regra em uma frase

Divida os dados primeiro. Depois, aprenda os parâmetros da transformação **somente no treino** e aplique a mesma transformação ao treino e ao teste.

Treino

1. `scaler.fit(X_train)`
2. `scaler.transform(X_train)`

Aprende média, desvio, mínimo ou máximo.

Teste

1. não executar `fit`
2. `scaler.transform(X_test)`

Usa os parâmetros aprendidos no treino.

Importante

Não se ajusta um scaler diferente no teste. Treino e teste usam a **mesma régua**, construída sem olhar o teste.

A Matriz de Aplicação Restrita

Métodos	Dados de Treino (O Passado)	Dados de Teste (O Futuro)
fit()	Aprende parâmetros fundamentais (ex: média, mediana, min/max).	**TOTALMENTE PROIBIDO.**
transform()	Aplica os parâmetros aprendidos para modificar os dados.	**OBRIGATÓRIO.** Usa a régua exata construída pelo treino para modificar o teste.
fit_transform()	Atalho eficiente para aprender e modificar simultaneamente.	**TOTALMENTE PROIBIDO.** Causa Data Leakage imediato.

Nota: Esta matriz define as permissões rigorosas para evitar contaminação do conjunto de teste.

Gemini Notebook

Por quê? Para evitar data leakage

Com vazamento

O pré-processamento aprende estatísticas usando todos os dados. Assim, o conjunto de teste influencia o treinamento antes da avaliação.

- a estimativa deixa de representar dados desconhecidos;
- a métrica pode ficar otimista;
- o erro pode permanecer invisível em um experimento.

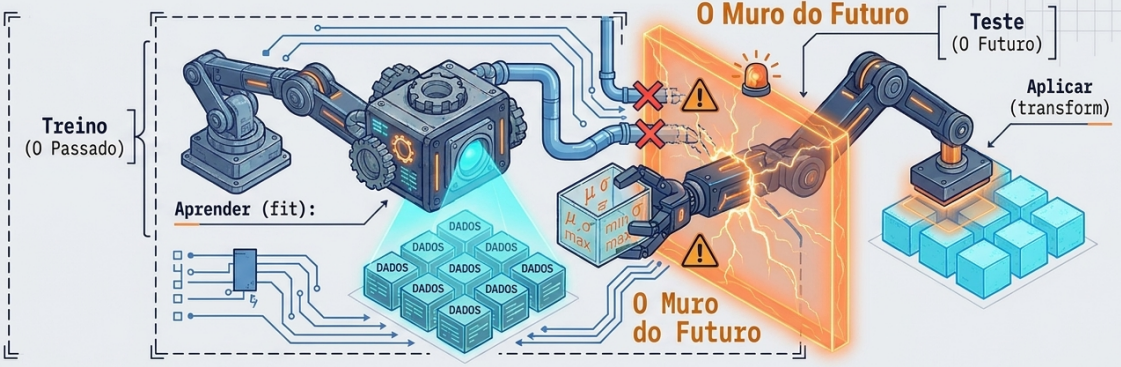
Sem vazamento

$$X_{tr}, X_{te} \longrightarrow \begin{cases} \texttt{fit}(X_{tr}) \\ \texttt{transform}(X_{tr}) \\ \texttt{transform}(X_{te}) \end{cases}$$

Teste = simulação do futuro

Ele só participa quando o fluxo já está definido e treinado.

O Pecado Capital: Vazamento de Dados (Data Leakage)



⚠ O Problema

Permitir que a etapa de pré-processamento "espie" informações do conjunto de teste antes da avaliação final.

⚠ O Sintoma Perigoso

O código executa perfeitamente e a acurácia é altíssima no ambiente de laboratório.

⚠ A Realidade

O erro permanece invisível até a produção. A estimativa deixa de representar dados desconhecidos, pois o modelo já "colou" na prova.

Gemini Notebook

Roteiro da aula alinhado ao plano de ensino

Semana 4

Pré-processamento de dados; laboratório; TDE; entrega da proposta do projeto.

1. separar antes de aprender: `fit`, `transform` e teste independente;
2. comparar Min–Max, padronização e alternativas para outliers;
3. tratar ausentes e categorias com `ColumnTransformer` e `Pipeline`;
4. escolher a divisão correta para classificação, tempo e grupos;
5. aplicar o fluxo completo ao projeto da equipe.

Quatro práticas intercaladas

1. corrigir leakage 2. comparar scalers 3. montar um pipeline 4. integrar toda a aula

`fit` aprende; `transform` apenas aplica

`fit(X_train)`

Aprende parâmetros exclusivamente no treino:

- média e desvio;
- mínimo e máximo;
- mediana;
- categorias.

`transform(X)`

Aplica parâmetros já aprendidos. Não recalcula estatísticas.

`fit_transform(X)`

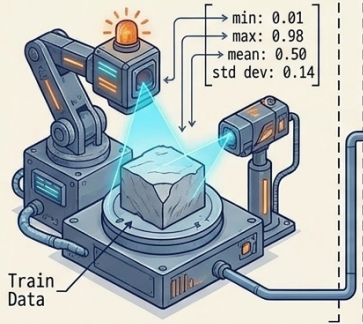
Executa as duas operações em sequência. Use no treino, nunca no teste.

Regra curta

`fit` e `fit_transform`: somente treino. `transform`: treino, validação, teste e novos dados.

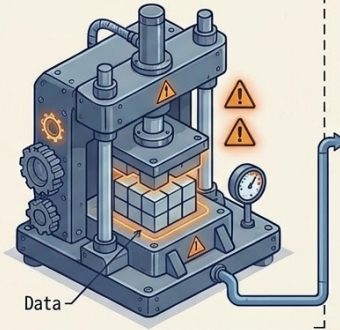
O Cérebro e a Prensa: Compreendendo Fit e Transform

Machine 1: O Scanner (fit)



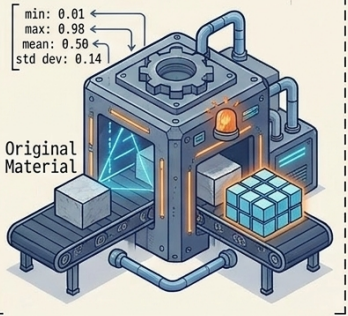
Extrai as medidas exatas do material. Ele APRENDE, mas não altera a forma física.

Machine 2: A Prensa (transform)



Aplica as medidas aprendidas para MOLDAR fisicamente os dados em um novo formato.

Machine 3: O Sistema Integrado (fit_transform)



Uma única ação: escaneia e molda simultaneamente o material original.

⚠ **Nota Técnica:** As operações são distintas para evitar o vazamento de dados entre Treino e Teste (ver slides anteriores).

Gemini Notebook

A divisão treino/teste vem antes do pré-processamento

Ordem incorreta

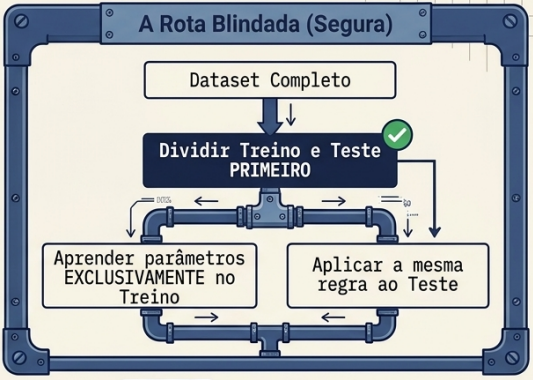
Dataset
↓
ajustar imputador/encoder/scaler
↓
dividir treino e teste

Ordem correta

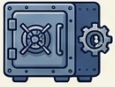
Dataset
↓
dividir treino e teste
↓
ajustar tudo somente no treino

O teste pode ser **transformado**; não pode ser usado para **aprender a transformação**.

A Regra de Ouro da Linha de Montagem



O modelo aprende estatísticas do futuro.



Segurança total contra vazamento.

O Princípio: O conjunto de teste representa o futuro desconhecido. Ele não participa de nenhuma decisão de transformação.

Gemini Notebook

Um único valor de teste já muda a transformação

Considere uma feature com treino $[20, 30, 40]$ e um valor de teste igual a 100.

Parâmetros aprendidos em	média	desvio padrão	z do teste 100
somente treino	30,0	8,16	8,57
treino + teste (leakage)	47,5	31,91	1,65

O que vazou?

O valor futuro alterou a média e o desvio usados para representar os próprios dados de treino. O modelo foi preparado com informação sobre o teste.

Código incorreto – o teste participa do scaler

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop(columns="target")
y = df["target"]

# ERRADO: aprende media e desvio usando todo X
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.25, random_state=42
)
```

Sintoma perigoso

O código executa normalmente. O problema não é sintático; é experimental.

Código correto – aprender no treino, aplicar no teste

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)

modelo.fit(X_train_s, y_train)
pred = modelo.predict(X_test_s)
```

A mesma transformação

`X_test` usa a média e o desvio aprendidos em `X_train`. Não é necessário – nem permitido – ajustar outro scaler no teste.

Atividade prática 1 – caça ao vazamento

```
X = df[["idade", "salario"]]
y = df["comprou"]

X["idade"] = X["idade"].fillna(
    X["idade"].median()
)

X = StandardScaler().fit_transform(X)

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=.25,
                     random_state=42)
```

Em duplas

1. Marque tudo o que aprende informação.
2. Indique onde o teste foi usado.
3. Reordene as operações.
4. Explique por que a mediana também pode vazar.

Tempo

8 minutos + discussão coletiva.

Solução da atividade 1 – o imputador também aprende

```
from sklearn.impute import SimpleImputer

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=.25, random_state=42
)

imp = SimpleImputer(strategy="median")
X_train_i = imp.fit_transform(X_train)
X_test_i  = imp.transform(X_test)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train_i)
X_test_s  = scaler.transform(X_test_i)
```

Generalização

Qualquer etapa que calcula estatísticas, seleciona atributos ou descobre categorias deve ser ajustada no treino.

Normalização e padronização não são sinônimos

Normalização Min–Max

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

No treino, normalmente leva os valores ao intervalo $[0, 1]$.

Padronização Z-score

$$z = \frac{x - \mu}{\sigma}$$

No treino, produz média próxima de 0 e desvio próximo de 1.

Ambas preservam a ordem dos valores

Elas mudam magnitude e unidade, mas não corrigem automaticamente assimetria, outliers ou erros de medição.

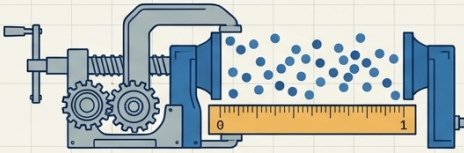
Calibrando os Instrumentos Numéricos

O Risco da Escala:
Modelos baseados em distância (KNN, SVM) são cegados por números grandes. Uma feature como Salário esmaga a influência da Idade.

Minmaxcaler

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

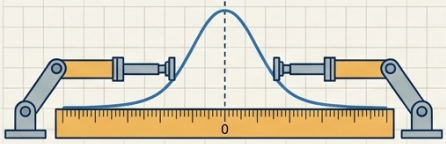
Mecânica: Espreme os dados para o intervalo exato de [0, 1]. O menor valor vira 0, , o maior vira 1.



StandardScaler (Z-Score)

$$z = \frac{x - \mu}{\sigma}$$

Mecânica: Força a média para 0 e o desvio padrão para 1. Mantém a dispersão, mas equaliza o idioma numérico.



Gemini Notebook

MinMaxScaler – o intervalo é aprendido no treino

Parâmetros armazenados

- `data_min_`
- `data_max_`
- `data_range_`
- `scale_`
- `min_`

Valor de teste fora do intervalo

Se o teste contém um valor menor que o mínimo de treino ou maior que o máximo, a transformação pode produzir valor abaixo de 0 ou acima de 1.
Isso não é leakage: é um sinal de que o novo dado extrapola o intervalo observado no treino.

Fonte: scikit-learn 1.9.0, documentação de `MinMaxScaler`.

StandardScaler – centro e dispersão vêm do treino

Parâmetros armazenados

- `mean_`: média de cada feature;
- `var_`: variância de cada feature;
- `scale_`: escala usada na transformação.

O que não significa

- não torna a distribuição normal;
- não remove outliers;
- não limita valores a um intervalo;
- não substitui análise exploratória.

Atenção

Média e desvio são sensíveis a valores extremos; por isso o resultado depende da qualidade do treino.

Outliers podem comprimir a maior parte dos dados

MinMaxScaler

Um extremo amplia o intervalo e comprime os demais valores.

StandardScaler

Um extremo desloca média e aumenta desvio padrão.

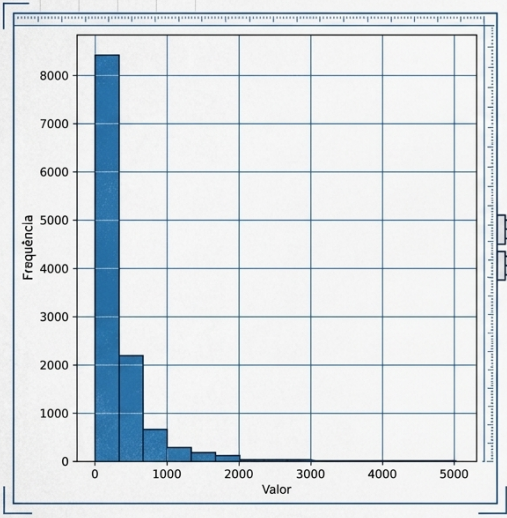
RobustScaler

Usa mediana e intervalo interquartil. Pode ser mais estável quando os extremos são legítimos.

Não escolha automaticamente

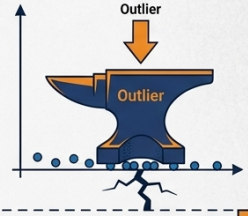
Primeiro verifique se o outlier é erro, evento raro importante ou parte esperada do fenômeno. Depois compare métodos dentro de um pipeline válido.

O Efeito Destrutivo das Anomalias



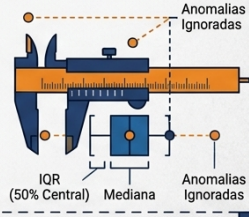
O Ponto de Falha

Média, desvio padrão, mínimo e máximo são extremamente sensíveis a outliers.
Um único erro de medição age como um peso gigante, esmagando o restante dos dados em um `MinmaxScaler` para zero, ou deslocando violentamente a média no `StandardScaler`.



A Solução Blindada (`RobustScaler`)

Ignora os extremos matemáticos. Utiliza a Mediana (o centro imperturbável) e o Intervalo Interquartil (IQR), focando sua régua exclusivamente onde repousam os 50% dos dados centrais. Anomalias são ignoradas pelo mecanismo de calibração.



Gemini Notebook

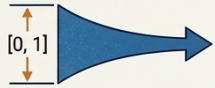
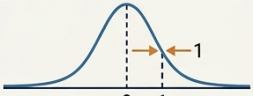
Alguns algoritmos sentem muito mais a escala

Família / algoritmo	Sensibilidade	Por quê?
KNN, K-means	alta	dependem diretamente de distâncias
SVM com kernel RBF	alta	distâncias determinam similaridade
Regressão logística, Ridge, Lasso	média/alta	otimização e regularização dependem da magnitude
Redes neurais	alta	escala afeta gradientes e convergência
PCA	alta	componentes dependem da variância
Árvores e Random Forest	baixa	divisões dependem principalmente da ordem dos valores

Consequência prática

Teste o pré-processamento junto com o modelo. Não avalie o scaler isoladamente.

Matriz de Diagnóstico: Qual Scaler Usar?

	Minmaxcaler	StandardScaler	RobustScaler
Mecanismo	 Intervalo fixo [0, 1].	 Média 0, Desvio Padrão 1.	Mediana e Intervalo Interquartil (IQR).
Sensibilidade a Outliers	Altíssima (Fatal)	Alta	Imune
Uso Ideal	Algoritmos que exigem limites rígidos (ex: Redes Neurais específicas) e dados onde se tem certeza da ausência de anomalias extremas.	Padrão da indústria para regressão linear, SVM e PCA, quando a distribuição original não é severamente distorcida por caudas longas.	Datasets complexos do mundo real contendo anomalias legítimas, ruídos de sensores ou assimétricas severas que não podem ser deletadas.

Gemini Notebook

Atividade prática 2 – compare três fluxos no Wine

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.linear_model import LogisticRegression

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=.25, random_state=42, stratify=y)

modelos = {
 "sem escala": LogisticRegression(max_iter=5000),
 "standard": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000)),
 "minmax": make_pipeline(MinMaxScaler(), LogisticRegression(max_iter=5000))
}
for nome, modelo in modelos.items():
    scores = cross_val_score(modelo, Xtr, ytr, cv=5)
    print(nome, scores.mean(), scores.std())
```

Regra da prática

Compare as opções apenas no treino. Guarde `Xte` e `yte` para uma avaliação final, depois da escolha.

Discussão da atividade 2 – o número é o começo da análise

Executar

1. Compare média e desvio na validação cruzada.
2. Troque `random_state`.
3. Adicione `RobustScaler`.
4. Observe avisos de convergência.

Explicar

1. Qual fluxo foi mais consistente no treino?
2. O resultado muda com outras divisões?
3. Por que não escolher usando o teste?
4. Onde o pipeline impede leakage?

Entrega rápida

Uma tabela com método, média, desvio e uma interpretação de duas frases.

Leakage não acontece apenas na normalização

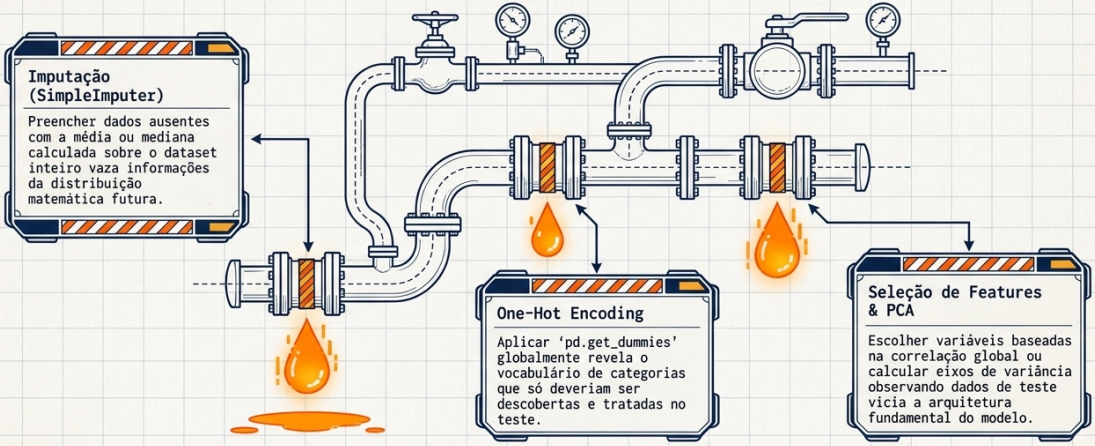
Etapa	O que aprende	Risco quando usa todo o dataset
Imputação	média, mediana ou moda	teste altera o valor de preenchimento
One-Hot Encoding	categorias existentes	teste revela categorias futuras
Seleção de features	relação entre X e y	target do teste influencia as features
PCA	direções de maior variância	teste altera o novo espaço
Remoção de baixa variância	variância das colunas	teste decide quais colunas sobrevivem
Transformações por quantis	distribuição empírica	teste participa dos quantis

Mesma regra

Divida primeiro. Ajuste cada transformação no treino. Aplique-a sem reajuste ao teste.

[As Fissuras Ocultas no Pré-Processamento]

Data Leakage não ocorre apenas na escala. Qualquer ferramenta que extraia estatísticas do dataset inteiro antes do split contamina o teste.



Imputação (SimpleImputer)

Preencher dados ausentes com a média ou mediana calculada sobre o dataset inteiro vaza informações da distribuição matemática futura.

One-Hot Encoding

Aplicar 'pd.get_dummies' globalmente revela o vocabulário de categorias que só deveriam ser descobertas e tratadas no teste.

Seleção de Features & PCA

Escolher variáveis baseadas na correlação global ou calcular eixos de variância observando dados de teste vicia a arquitetura fundamental do modelo.

Gemini Notebook

OneHotEncoder precisa prever categorias desconhecidas

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")

X_train_cat = encoder.fit_transform(X_train[["cidade"]])
X_test_cat  = encoder.transform(X_test[["cidade"]])
```

Exemplo

Treino: Curitiba, Londrina, Maringá. Teste: Ponta Grossa.
Com `handle_unknown="ignore"`, a categoria nova não cria uma coluna aprendida a partir do teste; ela é transformada de acordo com a regra definida no treino.

Evite

Aplicar `pd.get_dummies` separadamente no treino e no teste pode produzir colunas incompatíveis. Aplicar no dataset inteiro antecipa as categorias do teste.

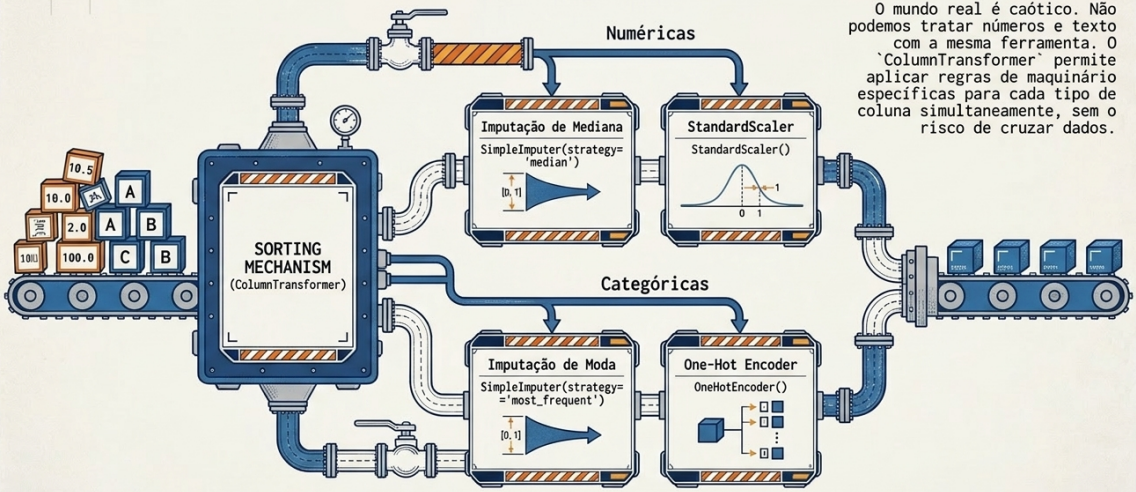
ColumnTransformer trata cada tipo de coluna corretamente

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

num_cols = ["idade", "salario"]
cat_cols = ["cidade"]

num_pipe = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler()
)
cat_pipe = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore")
)
pre = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("cat", cat_pipe, cat_cols)
])
```

O Maestro da Triagem: ColumnTransformer



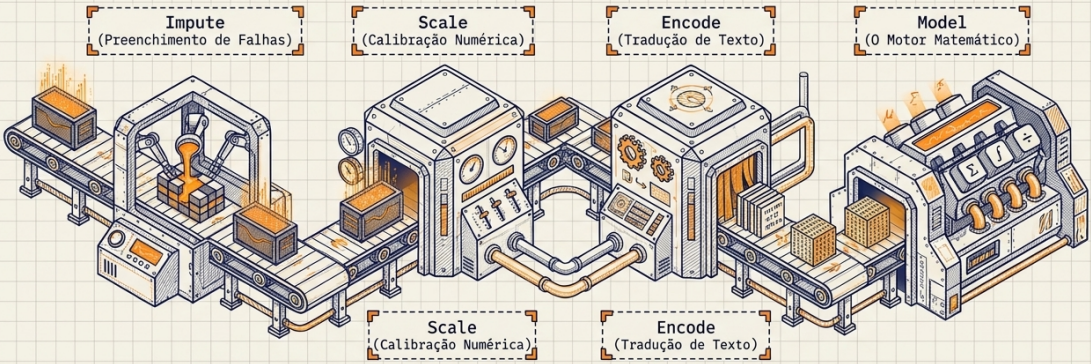
O mundo real é caótico. Não podemos tratar números e texto com a mesma ferramenta. O `ColumnTransformer` permite aplicar regras de maquinário específicas para cada tipo de coluna simultaneamente, sem o risco de cruzar dados.

Gemini Notebook

Pipeline transforma o fluxo em um único estimador

- `pipeline.fit(X_train, y_train)` ajusta transformadores e modelo no treino.
- `pipeline.predict(X_test)` apenas transforma o teste e realiza a previsão.
- O mesmo objeto pode ser usado em validação cruzada e ajuste de hiperparâmetros.
- O fluxo reduz divergências entre treinamento e uso futuro.

A Linha de Montagem Blindada: Pipeline



O `Pipeline` unifica as etapas de pré-processamento e o algoritmo de Machine Learning em um ÚNICO objeto estimador. Ele automatiza o fluxo, garante a reprodutibilidade absoluta e, mais importante, atua como um cofre de segurança que impede que o Teste sofra um `fit()` acidental por intervenção manual.

Gemini Notebook

Pré-processamento e modelo no mesmo pipeline

```python
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("preprocessamento", pre),
    ("modelo", LogisticRegression(max_iter=2000))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=.25, random_state=42, stratify=y
)

pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)
print(pipe.score(X_test, y_test))
```

Um objeto para salvar e reutilizar

O pipeline guarda o imputador, o scaler, o encoder e o modelo já ajustados.

O que acontece dentro de `fit` e `predict`?

`pipe.fit(X_train, y_train)`

1. imputadores aprendem estatísticas;
2. scaler aprende escala;
3. encoder aprende categorias;
4. transformações são aplicadas;
5. modelo aprende com o resultado.

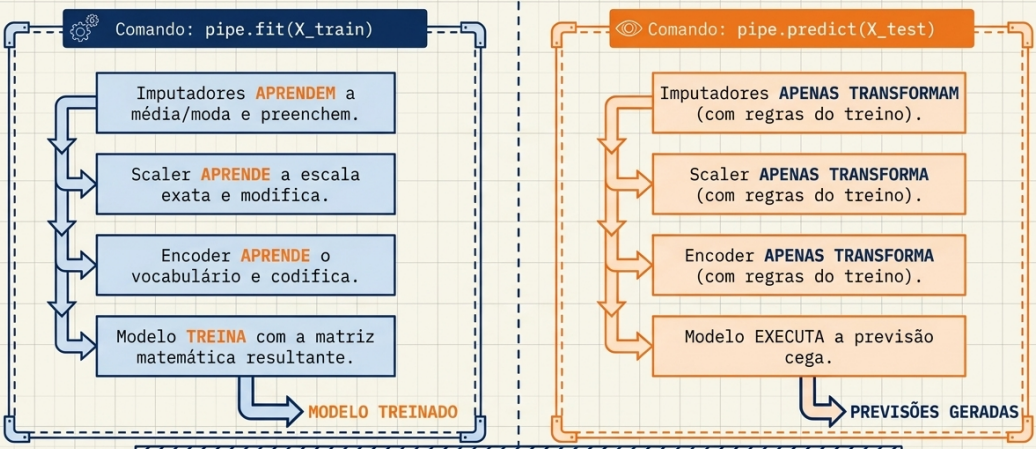
`pipe.predict(X_test)`

1. imputadores apenas transformam;
2. scaler apenas transforma;
3. encoder apenas transforma;
4. modelo prevê.

O teste não chama `fit`

É essa separação que o pipeline ajuda a garantir de forma consistente.

A Anatomia Interna do Pipeline



A Magia da Arquitetura: O Teste flui pelo mesmo encanamento, mas o sistema impede mecanicamente que qualquer etapa invoque um recalculo estatístico.

Gemini Notebook

Atividade prática 3 – transforme o código da equipe em pipeline

```python
# 1. liste as colunas por tipo
num_cols = [...]
cat_cols = [...]

# 2. construa pre
pre = ColumnTransformer([...])

# 3. una pre + modelo
pipe = Pipeline([
  ("pre", pre),
  ("model", LogisticRegression(max_iter=2000))
])

# 4. ajuste somente no treino
pipe.fit(X_train, y_train)
print(pipe.score(X_test, y_test))
```

Checklist da dupla

1. O target ficou fora de X?
2. Há identificadores inúteis?
3. Numéricas e categóricas estão separadas?
4. Ausentes são tratados no pipeline?
5. Existe algum `fit` antes do split?

Tempo

15 minutos. Use o dataset da equipe ou o DataFrame da Aula 03.

Treino, validação e teste têm funções diferentes

Treino

Aprende transformações e parâmetros do modelo.

Validação

Compara alternativas: scaler, modelo e hiperparâmetros.

Teste

Estima o desempenho final após todas as escolhas.

O teste é usado uma vez no final

Escolher o melhor modelo repetidamente com base no teste transforma o próprio teste em validação.

Exemplo de divisão – não é regra universal

70% treino, 15% validação e 15% teste; ou treino/teste com validação cruzada apenas dentro do treino.

Classificação: preservar proporções com `stratify`

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

Quando ajuda?

Em classificação, principalmente com classes desbalanceadas, a estratificação busca manter proporções semelhantes de cada classe no treino e no teste.

`random_state` não corrige leakage

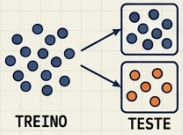
Ele torna a divisão reproduzível. A validade depende da ordem das operações e da estratégia de split apropriada ao problema.

Matriz de Estratégias de Separação

Dados Independentes / Padrão

↳ Divisão Aleatória (`train_test_split`)

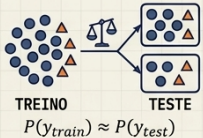
A estratégia básica e funcional. Segura unicamente quando não há absolutamente nenhuma correlação ou relação estrutural entre as diferentes linhas do dataset.



Classificação Desbalanceada

↳ Divisão Estratificada (`stratify=y`)

Mecanismo que força a manutenção das proporções numéricas exatas de cada classe do alvo (y) em ambos os conjuntos (Treino e Teste).



$$P(y_{train}) \approx P(y_{test})$$

Dados Cronológicos

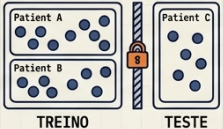
↳ Divisão Temporal (`TimeSeriesSplit`)

Respeita rigidamente a flecha do tempo. O algoritmo separa blocos contíguos do passado exclusivamente para prever blocos do futuro.

Observações Agrupadas (Ex: Pacientes)

↳ Divisão por Grupos (`GroupKFold`)

Garante matematicamente que a mesma entidade caia inteiramente no Treino ou inteiramente no Teste, bloqueando o reconhecimento fraudulento.



Gemini Notebook

Nem todo problema permite um split aleatório

Estrutura dos dados	Risco do split aleatório	Estratégia
Série temporal	treinar com o futuro e testar no passado	divisão cronológica / `TimeSeriesSplit`
Várias linhas por paciente	mesmo paciente em treino e teste	split por grupo
Várias imagens por pessoa	identidade aparece nos dois lados	`GroupShuffleSplit` / `GroupKFold`
Imóveis do mesmo empreendimento	unidades quase duplicadas nos dois lados	agrupar por empreendimento
Medições repetidas de sensor	janelas vizinhas muito semelhantes	separar por período, sessão ou equipamento

Pergunta de projeto

Quais observações compartilham informação e, por isso, não deveriam ser separadas entre treino e teste?

O Ponto Cego da Divisão Aleatória

A função `**train_test_split(random_state=42)**` não é universalmente segura. Em estruturas de dados complexas, a mistura aleatória causa falhas catastróficas por vazamento estrutural.

⚠ Séries Temporais

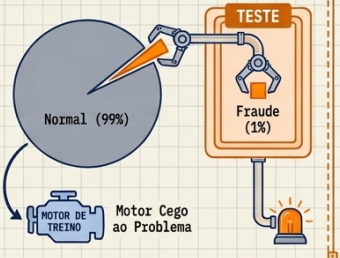
Misturar o tempo permite que o modelo treine com dados do futuro para prever o passado. Cria um Vazamento Temporal absurdo.

⚠ Pacientes / Entidades Únicas

Múltiplos registros da mesma pessoa divididos aleatoriamente caem em ambos os lados. O modelo não aprende a prever o quadro clínico, apenas reconhece o paciente especifico.

⚠ Classes Desbalanceadas

Uma divisão cega pode, por azar, jogar todos os raros exemplos de fraude (1%) inteiramente para o teste, deixando o motor de treino completamente cego ao problema.



Gemini Notebook

Quatro famílias de vazamento

1. Pré-processamento

Scaler, imputador, encoder, PCA ou seleção ajustados com o teste.

2. Target leakage

Uma feature contém a resposta ou informação disponível somente depois do evento.

3. Vazamento temporal

Dados futuros ajudam a prever o passado.

4. Vazamento por grupos/duplicatas

Registros da mesma entidade aparecem dos dois lados da divisão.

Acurácia alta demais é um sinal, não uma prova

O diagnóstico exige conhecer como e quando cada feature foi produzida.

TDE e projeto – entrega da proposta em 27/08/2026

Objetivo da entrega

Demonstrar que a equipe possui um problema de ML bem formulado, um dataset viável e um plano de pré-processamento que permita avaliação sem vazamento.

- consolidar problema, observação, features e target;
- apresentar fonte, tamanho e limitações do dataset;
- mostrar uma EDA inicial com ausentes, outliers e categorias;
- definir a estratégia de divisão dos dados;
- propor o pipeline inicial;
- registrar riscos de leakage específicos do projeto.

Não basta entregar código executando

A proposta precisa justificar as decisões com base no significado dos dados.

Estrutura mínima da proposta

1. **Título e equipe**
2. **Problema**: uma frase clara e a decisão que a previsão apoia
3. **Dataset**: fonte, licença, observação, período, linhas e colunas
4. **Target**: tipo, unidade, classes e disponibilidade temporal
5. **Features**: significado e momento em que ficam disponíveis
6. **EDA inicial**: distribuições, ausentes, duplicatas, outliers e categorias
7. **Divisão**: aleatória, estratificada, temporal ou por grupos
8. **Pipeline**: imputação, codificação, escala e primeiro modelo
9. **Métrica**: adequada ao objetivo e ao balanceamento
10. **Riscos e próximos passos**: leakage, vieses, qualidade e dados faltantes

Formato sugerido

Notebook executável + proposta curta em PDF, com links para dataset e repositório.

Atividade prática 4 – atividade completa da aula

Parte A – problema e dados

1. Defina X, y e a métrica.
2. Escolha o split: aleatório, estratificado, temporal ou por grupos.
3. No treino, examine ausentes, outliers e categorias.
4. Liste tudo o que precisa de `fit`.

Parte B – pipeline e avaliação

1. Construa o `ColumnTransformer`.
2. Una pré-processamento e modelo em um pipeline.
3. Compare scalers usando somente o treino.
4. Avalie uma vez no teste e interprete o resultado.

Produto da aula

Notebook executável + tabela de resultados + um parágrafo: "Como nossa equipe evita vazamento de dados?"

Atividade prática 4 – template de implementação

```
X = df.drop(columns=TARGET)
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=.25, random_state=42,
    stratify=y if TIPO == "classificacao" else None)

pre = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"),
                          StandardScaler()), NUM_COLS),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), CAT_COLS)
])

pipe = Pipeline([("pre", pre), ("model", MODELO)])
pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)
resultado = METRICA(y_test, pred)
```

Adapte, não copie cegamente

Escolha scaler e hiperparâmetros no treino. Série temporal, grupos, regressão e classes desbalanceadas exigem escolhas específicas.

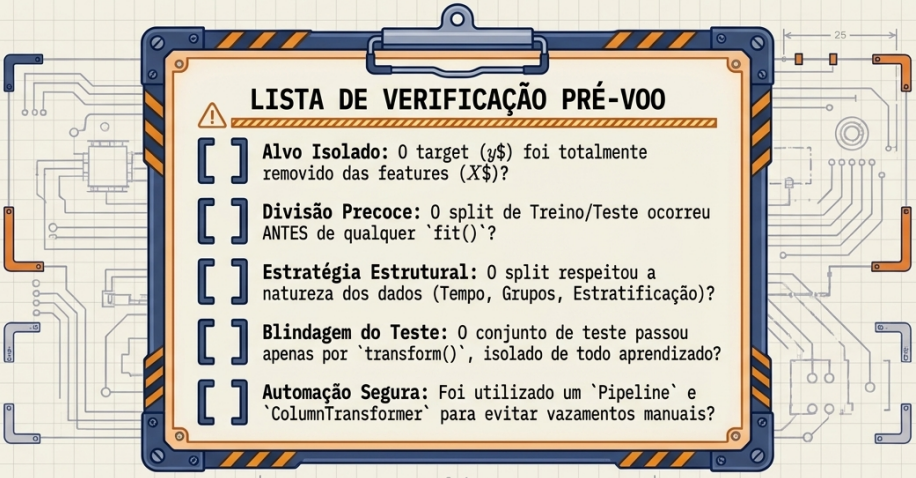
Checklist antes de confiar no resultado

- ☐ O target foi removido de X?
- ☐ As features existem no momento da previsão?
- ☐ A divisão respeita tempo e grupos?
- ☐ O teste ficou fora de todos os `fit`?
- ☐ Ausentes são tratados no pipeline?
- ☐ Categorias novas são previstas?
- ☐ O scaler é adequado ao modelo?
- ☐ Outliers foram analisados no contexto?
- ☐ A métrica corresponde ao problema?
- ☐ O teste foi usado somente no final?
- ☐ O fluxo é reproduzível?
- ☐ As decisões estão justificadas?

Regra final

Se uma etapa aprende algo, ela pertence ao treinamento – e deve estar dentro do pipeline.

O Checklist do Engenheiro de Machine Learning



REGRA FUNDAMENTAL: Se uma etapa da linha de montagem extrai e aprende qualquer parâmetro matemático dos dados, ela pertence estritamente ao bloco de treinamento.

Gemini Notebook

Resumo da Aula 04

Hoje consolidamos:

- divisão dos dados antes do pré-processamento;
- diferença entre `fit`, `transform` e `fit_transform`;
- MinMaxScaler, StandardScaler e efeito dos outliers;
- leakage em imputação, codificação, seleção e PCA;
- `ColumnTransformer` para dados heterogêneos;
- `Pipeline` para unir transformações e modelo;
- splits estratificados, temporais e por grupos;
- aplicação direta à proposta do projeto.

Próxima aula – Regressão

Modelos de regressão, função de custo, ajuste, previsão e métricas para targets contínuos.

Referências e documentação

- GÉRON, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. 3. ed. O'Reilly, 2022.
- JAMES, G. et al. *An Introduction to Statistical Learning: with Applications in Python*. 2. ed. Springer, 2023.
- SCIKIT-LEARN. Common pitfalls and recommended practices. Documentação estável 1.9.0.
- SCIKIT-LEARN. Preprocessing data.
- SCIKIT-LEARN. Pipelines and composite estimators.
- SCIKIT-LEARN. ColumnTransformer.
- SCIKIT-LEARN. Cross-validation: evaluating estimator performance.
- Materiais da disciplina: plano de ensino, cronograma, TDE, Aulas 01–03 e `ML-Atividades123.zip`.

Consulta

Documentação web consultada em 24 de agosto de 2026.

Perguntas?

Aprendizagem de Máquina – Aula 04